Module 4

Pointers

Part - II

Syllabus

- Declaration, Operations on pointers
- Passing pointer to a function
- Accessing array elements using pointers
- Processing strings using pointers
- Pointer to pointer
- **Array of pointers**
- **Pointer to function**
- **Pointer to structure**
- **Dynamic Memory Allocation**

Array of Pointers

Array of Pointers

An array of pointers stores the addresses of all the elements of the array

Declaration syntax

```
datatype * ptr[Max_size];
```

Initialization syntax

```
char *str_name[ ] = {“Str1",“Str2",“Str3“ };
```

WAP using Array of Pointers

```
#include<stdio.h>
int main()
{
   int a, b, c, i, *p[10];
   a = 6; b = 7; c = 8;
   p[0] = &a;
   p[1] = &b;
   p[2] = &c;
   for (i=0; i<3; i++)
       printf("%d  \t", *p[i]);
   return 0;
}
```

Output:

6 7 8

WAP using Array of Pointers

```
#include<stdio.h>
int main()
{
   int i, N, *ptr[10];
   int var[3] = {20, 30, 40};
   for(i=0; i<3; i++)
       ptr[i] = &var[i];
   printf("The address and variables are :\n");
   for(i=0; i<3; i++)
     printf("%p --- %d  \n", ptr[i],*ptr[i]);
   return 0;
}
```

Output:

The address and variables are :

0x7ffc06d3d554 --- 20

0x7ffc06d3d558 --- 30

0x7ffc06d3d55c --- 40

Array of pointer strings

- An array of string pointers stores the addresses of the strings present in the array.
- Each pointer points to the first character of the string or the base address of the string.

Example

```
char *sports[5] = { "golf", "hockey",
      "football", "cricket", "shooting" };
```

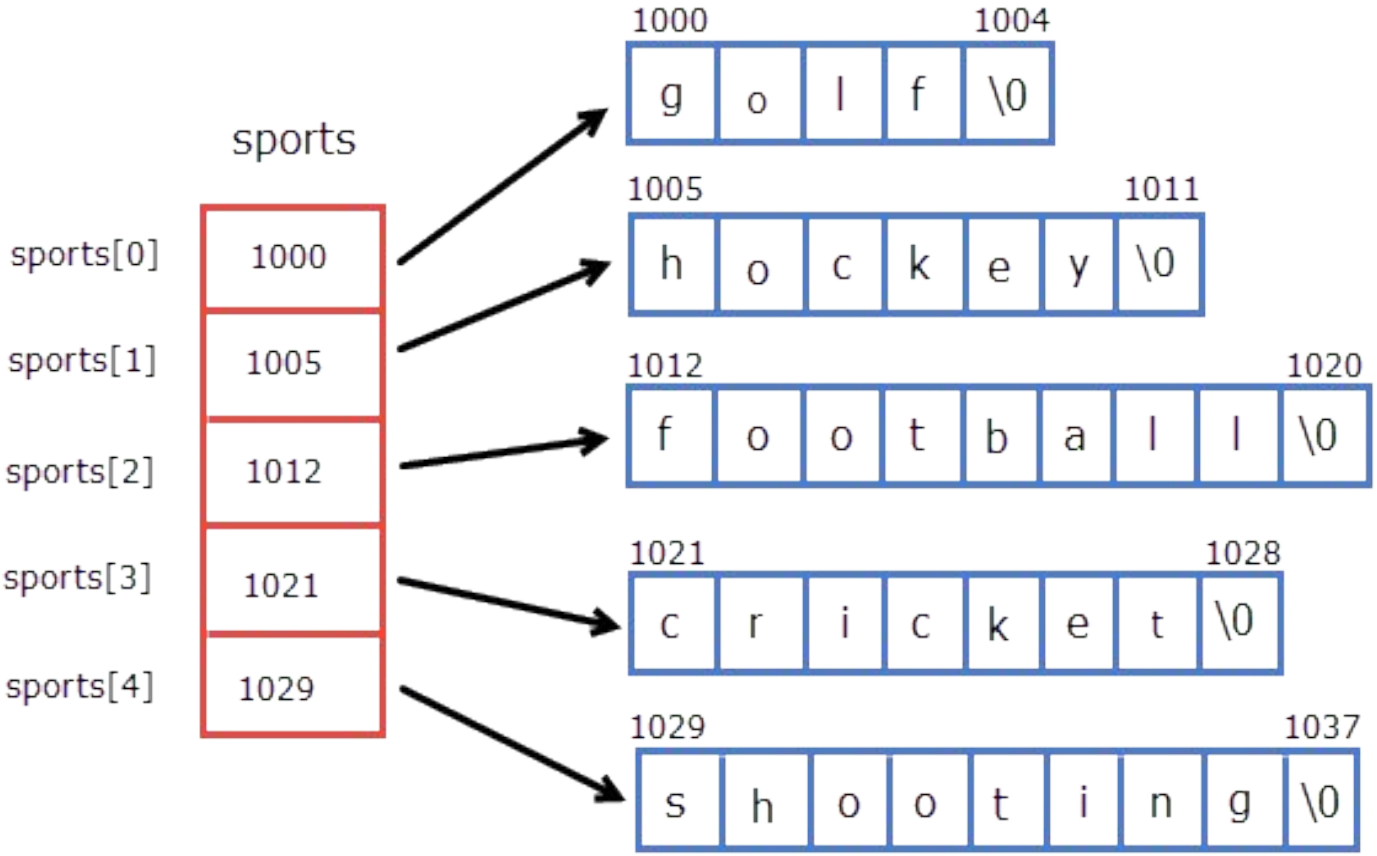
Recollect from Module 2- "2-D strings"

Memory representation of array of strings

`sports[5][10]`

1000	g	o	l	f	\0	\0	\0	\0	\0	\0
1010	h	o	c	k	e	y	\0	\0	\0	\0
1020	f	o	o	t	b	a	l	l	\0	\0
1030	c	r	i	c	k	e	t	\0	\0	\0
1040	s	h	o	o	t	i	n	g	\0	\0

Memory representation of array of pointer strings



Inference : Just by creating an array of pointers to string instead of array 2-D array of characters we are saving memory

WAP to demonstrate how to access strings in the array of pointers

```
#include<stdio.h>
int main()
{
int i;
char *name[3] = {"Messi", "Ronaldo", "Neymar"};
printf("The address and names are :\n");
for(i=0; i<3; i++)
  printf(" %p ---- %s  \n", name[i],name[i]);
return 0;
}
```

The address and names are :
0x57365e1b0004 ---- Messi
0x57365e1b000a ---- Ronaldo
0x57365e1b0012 ---- Neymar

Pointer as function argument

Recollect

Module 3- Functions

- Pass **variables** as arguments to function definition
- Pass **array** as arguments to function definition
- Pass **structure** as arguments to function definition

Module 4- Pointers

- Pass **pointers** as arguments to function definition

Pointer to function

WAP to find the sum of two numbers using pointers and function

```
void pointer_sum(int *, int *);
int main()
{
  int a, b, *pa, *pb
  a = 5; b = 4;
  pa = &a; pb = &b;
  pointer_sum(pa, pb); // pointer_sum(&a, &b);
  return 0;
}

void pointer_sum(int *pa, int *pb)
{
  int sum;
  sum = *pa +*pb;
  printf("The sum of two r
```

Address of variable passed as arguments

Output:

The sum of two numbers is : 9

Sum of elements of array using pointer and user defined function

```
#include<stdio.h>
int sum_array(int N, int *ptr);
int main()
{
int N, sum, arr[100], *ptr;
ptr = arr;
printf("Enter the size of array : ");
scanf("%d", &N);
sum = sum_array(N, ptr);
printf("The sum of array elements is : %d", sum);
return 0;
}
```

Address of array passed as arguments

Sum of elements of array using pointer and user defined function

```
int sum_array(int N, int *ptr)
{
int i, arr[100], sum =0;
printf("Enter the elements of array :\n");
for(i=0;i<N;i++)
  {
   scanf("%d", ptr +i);
   sum = sum + *(ptr+i);
  }
  return(sum);
  }
```

Output:

Enter the size of array : 5

Enter the elements of array :

3 5 8 9 6

The sum of array elements is : 31

Pass by value (Call by value)

Pass by reference (Call by reference)

Pass by value

- When arguments are passed by value then the **copy** of the actual parameters is transferred from calling function to the called function definition in formal parameters.
- Now any changes made in the formal parameters in called function definition will **not** be reflected in actual parameters of calling function.
- In call by value, **different memory** is allocated for actual and formal parameters

Program – pass by value

```
void increment(int);
int main()
{
int x = 10;
printf("Before function call x is : %d\n", x);
increment(x);          // Variable passed as argument
printf("After function call x is : %d\n", x);
return 0;
}
void increment(int x)
{
x = x+1;
x++;
printf("  The value of x
}
```

Output:

Before function call x is : **10**

The value of x within function is : 12

After function call x is : **10**

Program – pass by value (Swapping)

```
void swap_value(int, int);
void main()
{
int a = 10, b = 25;
printf("Before calling swap function a is : %d, \t b is : %d\n",a,b);
swap_value(a,b);      // Variable passed as argument
printf("After calling swap function a is : %d, \t b is : %d\n",a,b);
}

void swap_value(int a,int b)
{
int temp;
temp = a; a = b; b = temp;
printf("The swapped values in fn defn a is :%d, \t b is : %d\n",a,b);
}
```

Output- pass by value (swapping)

Before calling swap function **a is : 10, b is : 25**

The swapped values in fn defn **a is :25, b is : 10**

After calling swap function **a is : 10, b is : 25**

Pass by reference

- When arguments are passed by reference then the **address** of the actual parameters is transferred from calling function to the called function definition in formal parameters.
- Now any changes made in the formal parameters in called function definition will be reflected in actual parameters of calling function.
- In call by reference, the **memory allocation is similar** for both formal parameters and actual parameters.

Pass by reference

```
void increment(int *);
int main()
{
int x , *px;
x = 10;
px = &x;
printf("Before function call x is : %d\n", x);
increment(px);      // Address (Pointer) passed as argument
printf("After function call x is : %d\n", x);
return 0;
}
void increment(int *px)
{
*px = (*px)+1;
(*px)++;
printf("The value of *px within function is : %d\n", *px);
}
```

Output – Pass by reference

Before function call x is : 10

The value of *px within function is : **12**

After function call x is : **12**

Pass by reference

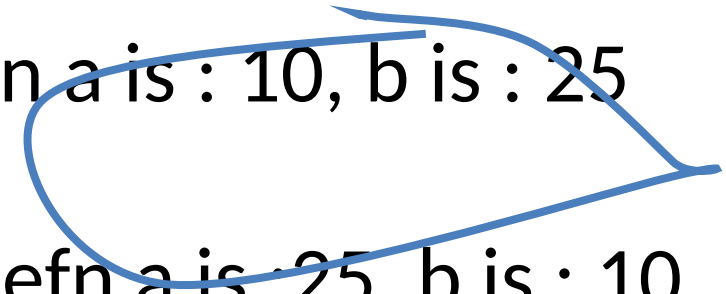
```
void swap_ref(int *, int *);
void main()
{
int a = 10, b = 25;
printf("Before calling swap function a is : %d , b is : %d\n",a,b);
swap_ref(&a,&b); // Address (Pointer) passed as argument
printf("After calling swap function a is : %d , b is : %d\n",a,b);
}

void swap_ref(int *pa,int *pb)
{
int temp;
temp = *pa; *pa = *pb; *pb = temp;
printf("The swapped values in FnDefn a is:%d ,b is : %d\n", *pa,*pb);
}
```

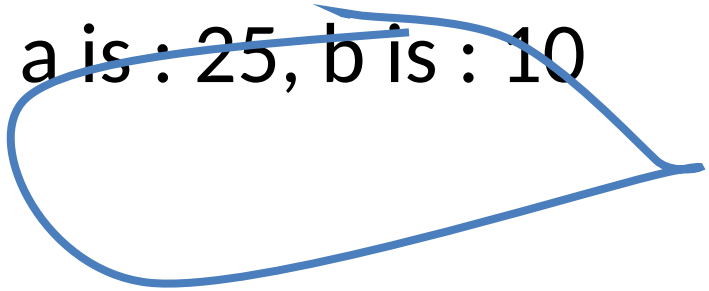
Output – Pass by reference (Swapping)

Before calling swap function a is : 10, b is : 25

The swapped values in FnDefn a is :25, b is : 10

After calling swap function a is : 25, b is : 10





Multiple return values using pointer

We can return more than one values from a function by using the method called

1) Call by address
2) Call by reference
3) Pass by reference (address)

Use pointer and user defined function to find the quotient and remainder

```
#include<stdio.h>
void div(int a, int b, int *quotient, int *remainder)
{
    *quotient = a / b;
    *remainder = a % b;
}
int main()
{
int a, b, q, r;
printf("Enter the numbers : ");
scanf("%d %d", &a, &b);
div(a, b, &q, &r);
printf("Quotient is: %d\n Remainder is: %d\n", q, r);
return 0;
}
```

Output:

Enter the numbers :

7 2

Quotient is: 3

Remainder is: 1

Note: Two values - Quotient and remainder are returned to main program

Use pointer and user defined function to find the area and perimeter

```
#include<stdio.h>
#define PI 3.14
void perimeter_area(float, float*, float*);
int main()
{
float r, area, perimeter;
printf("Enter the radius : ");
scanf("%f", &r);
perimeter_area(r, &area, &perimeter);
printf("The area of the circle is : %f\n", area);
printf("The perimeter of the circle is : %f", perimeter);
return 0;
}
```

Use pointer and user defined function to find the area and perimeter -contd

```
void perimeter_area(float r, float *area, float *perimeter)
  {
  *area = PI*r*r;
  *perimeter =  2*PI*r;
  }
```

Output:

Enter the radius : 10

The area of the circle is 314.000000

The perimeter of the circle is 62.799999

POINTER TO STRUCTURE

Pointer to structure

It is pointer variable that points to the address of a structure

Syntax

```
struct keyword Name_of_struct  *ptr_name
```

Initialization of structure pointer

```
struct Student
{
int id;
char name[30];
int age;
char gender[15];
};
struct Student student1;
struct Student *ptr = &student1;
```

Pointer to structure

It is pointer variable that points to the address of a structure

Example

```
struct inventory
{
    char name[30]
    int number;
    float price;
} product[2], *ptr
```

```
ptr  = product
```

assigns zeroth element of the product to ptr

Accessing Pointers to Structures

To access members of the structure using a pointer, we have two methods :

- Using Indirection Operator(*) and Dot (.) operator
- Using Arrow Operator (->)

Accessing Pointers to Structures

```
#include<stdio.h>
#include<string.h>
int main()
{
struct Employee
{
    int id;
    char name[100];
    int salary;
};

struct Employee employee1;
struct Employee *ptr;
```

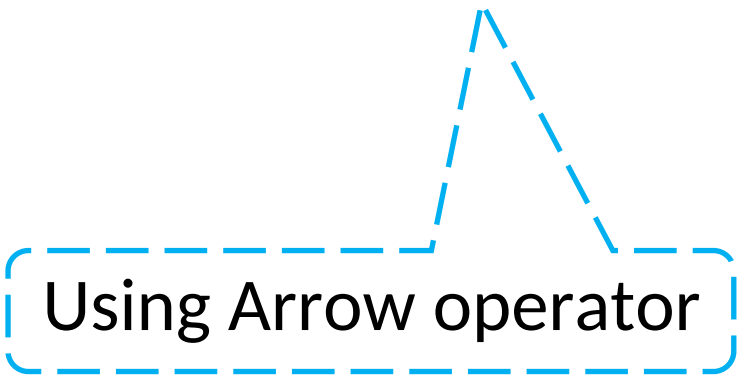
Accessing Pointers to Structures

```
ptr = &employee1;
employee1.id = 1;
employee1.salary = 10000;
strcpy(employee1.name , "Susan");
printf("Name is : %s\n",(*ptr).name);
printf("The ID is : %d\n", (*ptr).id);
printf("Salary is: %d", (*ptr).salary);
return 0;
}
```

Using Indirection Operator *, .

Accessing Pointers to Structures

```
printf("Name is : %s\n",ptr ->name);
printf("The ID is : %d\n", ptr-> id);
printf("Salary is: %d", ptr ->salary);
```



Output:

Name is : Susan

The ID is : 1

Salary is: 10000

Dynamic memory allocation

Dynamic memory allocation

- It is **allocating memory** while the **program is running**, instead of at the time of writing the code.
- It allows to request memory from the system as needed using functions like malloc(), calloc(), or realloc(), and release it using free().
- This gives flexibility to handle data whose size may not be known in advance, like user input or dynamic data structures such as linked lists.

Dynamic memory allocation

Function	Purpose	Initializes Memory?	Can Resize?	Requires free()?	Header File
`malloc()`	Allocates single block of memory	No	No	Yes	`<stdlib.h>`
`calloc()`	Allocates multiple blocks & zeroes them	Yes	No	Yes	`<stdlib.h>`
`realloc()`	Resizes previously allocated memory	No (old data kept)	Yes	Yes	`<stdlib.h>`
`free()`	Deallocates previously allocated memory	N/A	N/A	Itself releases	`<stdlib.h>`

malloc()

- The malloc() function stands for Memory Allocation.
- It is used to dynamically allocate a single block of memory of the specified size during the execution of the program.

calloc()

- The calloc() function stands for Contiguous Allocation.
- It is used to allocate memory for multiple elements and also initializes all of them to zero.

realloc()

- The realloc() function stands for Reallocation of Memory.
- It is used to resize an already allocated memory block during program execution—either to expand or shrink the existing block.

free()

- The free() function is used to release dynamically allocated memory back to the system.
- It helps prevent memory leaks and ensures efficient memory usage during the program's lifecycle.